Autoscaling — HPA (Horizontal Pod Autoscaler)

What is Autoscaling?

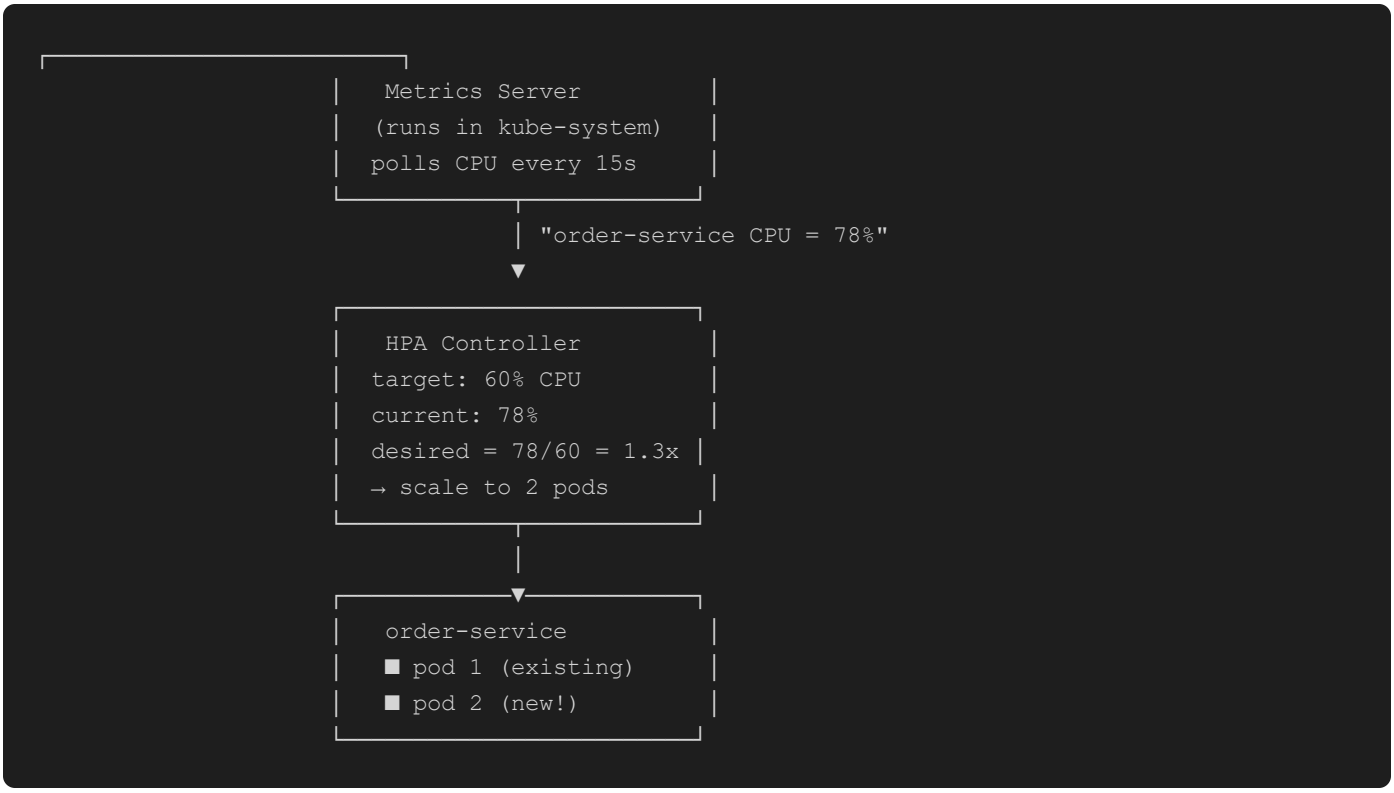
When traffic increases, Kubernetes automatically adds more copies (pods) of your service.

When traffic drops, it removes them. You pay only for what you use.

This is called **Horizontal Pod Autoscaling (HPA)** — "horizontal" means more pods,

not a bigger pod.

How HPA Works



Formula: `desired_replicas = ceil(current_replicas × current_metric / target_metric)`

Our HPA Configuration

From `k8s/services/deployments.yaml`, every microservice has:

```
---
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: order-service-hpa
  namespace: ecommerce
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: order-service
  minReplicas: 1       # never go below 1 (no cold starts)
  maxReplicas: 3       # never exceed 3 (resource limit)
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 60    # scale when avg CPU > 60%
```

Setting	Value	Why
`minReplicas: 1`	Always 1 pod	Pod is ready immediately for requests
`maxReplicas: 3`	Cap at 3 pods	1 pod per service = learning, not production scale
`averageUtilization: 60%`	Scale at 60% CPU	Buffer before saturation; 80% would leave no headroom

---

Timeline: What Happens During a Traffic Spike

```
Time 0s   → Traffic increases 3×
Time 15s  → Metrics Server reports: CPU = 75%
Time 30s  → HPA calculates: need 2 pods. Sends scale command to Deployment.
Time 60s  → New pod scheduled on a node
Time 90s  → New pod passes readiness probe (Spring Boot started)
Time 90s  → Kong starts sending traffic to both pods (load balanced)
Time ...  → Traffic drops. HPA waits 5 minutes before scaling DOWN
            (scaleDown stabilization window — avoids flapping)
Time +5m  → HPA removes extra pod
```

Why 5 minutes before scale-down? Traffic often spikes and drops repeatedly. Removing a pod immediately and adding it back 30s later is wasteful and causes latency. The stabilization window prevents this "flapping."

Load Balancing Between Pods

Kong (the API Gateway) uses **round-robin** load balancing by default.

When order-service scales from 1 to 2 pods, Kong's upstream automatically picks up the new pod IP (via Kubernetes service DNS).

```
Client → Kong :8000 → order-service (K8s Service)
                          ├── pod-1 :8083  ← request 1
                          ├── pod-2 :8083  ← request 2
                          └── pod-1 :8083  ← request 3 (round-robin)
```

The **Kubernetes Service** (ClusterIP) is the stable DNS name (`order-service.ecommerce.svc.cluster.local`).

Kong always calls this DNS — Kubernetes itself load-balances across whichever pods are healthy.

Testing Autoscaling

```
# Watch HPA status live
kubectl get hpa -n ecommerce --watch

# Generate load on order-service
kubectl run load-test --image=busybox --rm -it --restart=Never -- \
  sh -c "while true; do wget -q -O- http://order-service.ecommerce.svc.cluster.local:8083/api/
orders/user/1; done"

# In another terminal — watch pods scale
kubectl get pods -n ecommerce --watch

# Check CPU usage
kubectl top pods -n ecommerce
```

Expected output:

```
NAME                        READY   STATUS    REPLICAS
order-service-hpa           1/3     Running   1        ← initial
order-service-hpa           1/3     Running   2        ← after spike
order-service-hpa           1/3     Running   1        ← after quiet period
```

On Real AWS — What Changes

In production EKS, autoscaling has two more layers:

```
HPA              — adds/removes pods within existing nodes
Cluster Autoscaler — adds/removes EC2 nodes when pods can't be scheduled
  (triggered when: HPA wants 4 pods, but only 3 fit on existing nodes)

AWS Cost Implication:
  m5.large = ~$0.096/hr
  r5.large = ~$0.126/hr
  With our maxReplicas=3, worst case = 18 pods across ~6 nodes ≈ $0.60/hr
```

In Floci, the node groups are simulated — the HPA still runs and scales pods,

but there's no actual EC2 being added (k3s has unlimited virtual capacity).

Resource Requests vs Limits

Every pod in `k8s/services/deployments.yaml` has:

```
resources:
  requests:
    cpu: 250m          # guaranteed: 0.25 CPU core
    memory: 512Mi      # guaranteed: 512MB RAM
  limits:
    cpu: 500m          # max: 0.5 CPU core (throttled if exceeded)
    memory: 1Gi        # max: 1GB RAM (OOM-killed if exceeded)
```

Why this matters for HPA: HPA calculates "% of request" not "% of node." If a pod uses 300m CPU and its request is 250m → usage = 120% → HPA scales immediately. Setting realistic requests is critical for HPA to work correctly.